

Hangman. In this task, the agent acts as the host, privately selecting a secret word while the user guesses letters (see Appendix Figure 1 a). The Player issues guesses using a seeded policy that balances frequency-weighted sampling (prioritizing high-probability English letters like ‘e’, ‘t’, ‘a’) with exploration. After each guess, the agent must update the public state, displaying the current word pattern (e.g., “_ a _ e _”) and the list of guessed letters. At the fork point t_{fork} , we extract these hard constraints (word length, revealed letter positions, and absent letters) and filter the Wordfreq database (Speer, 2022) to generate a set of candidate words that are syntactically indistinguishable from the secret given the public history; if the database yields insufficient candidates, we supplement the set with LLM-generated proposals.

Diagnosis Simulator. This task evaluates consistency in the medical domain (see Figure 1 b), and it has been designed to emulate a medical student practicing differential diagnosis with an LLM. In the interaction the agent acts as the patient holding a hidden ground-truth condition, while the Player questions them to narrow down possibilities. Utilizing the DDXPlus dataset (Fansi Tchango et al., 2022), the Player issues yes/no questions about specific symptoms (evidences) to prune the hypothesis space. At t_{fork} , we generate the candidate set by filtering the dataset for all conditions that remain logically consistent with the accumulated positive and negative evidence (e.g., Fever=Yes, Cough=No). As before, this set is deduplicated and, if the database yields insufficient candidates, we supplement the set with LLM-generated proposals.

Evaluation. To assess the agent’s performance, we spawn one dialogue branch for each candidate secret and ask the agent to confirm or deny it (e.g., ‘Is the secret word “word”?’); we explicitly instruct the agent to output only ‘yes’ or ‘no’ to facilitate deterministic parsing. We first identify cases of **Leakage**, where the secret was explicitly revealed in the public transcript prior to the reveal turn (detected via case-insensitive matching). These are strictly categorized as failures of public underdetermination. For the remaining non-leaking runs, we measure **Self-Consistency**, defined as the rate at which the agent consistently affirms its revealed secret across branches without affirming any other candidate. We categorize the remaining failures into three behaviors: **Over-Confirmation**, where the agent correctly affirms the revealed secret but also falsely affirms at least one alternative candidate; **State Substitution**, where the agent denies the revealed secret but affirms at least a different candidate; and **All Denial**, where the agent rejects every proposed candidate.

The full prompt specifications for both tasks are provided in Appendix G.

3.1 COMPARATIVE BASELINES.

We compare our proposed methods against several baselines. We first consider Vanilla Stateless LLMs (standard POCAs), which generate responses solely from the public dialogue history. When using reasoning models (Section 4), we explicitly discard their internal reasoning traces to enforce the POCA setting. To establish an upper bound, we also evaluate Private Chain-of-Thought (CoT) agents that retain all hidden reasoning across turns. This baseline shows that persistent hidden traces are sufficient in principle to solve PSITs, but it is not a practical deployment solution: reasoning traces are highly verbose, causing the token budget to grow rapidly, precisely why standard chat interfaces discard them. Our aim is therefore to match this hidden-state benefit with a bounded and controllable memory representation. A cost analysis is provided in Section 5.

Agentic Memory Baselines. We compare against state-of-the-art memory-augmented agents that rely on external persistent textual memory to support generation. Although these systems are designed for long-term retrieval and conversational coherence rather than private state maintenance, they provide the closest existing baselines for evaluating whether current memory designs can implicitly support hidden state. In all cases, we use the baselines’ official libraries and native memory update and retrieval APIs, adapting only their interface to the shared dialogue setting.

A-mem organizes interactions into atomic, linked notes inspired by the Zettelkasten method, with LLM-driven semantic linking and memory updates (Kadavy, 2021; Xu et al., 2025).

Mem0 maintains long-term consistency via LLM-based extraction and dynamic ADD/UPDATE/DELETE operations over stored facts (Chhikara et al., 2025).

LightMem follows a hierarchical memory architecture with sensory, short-term, and long-term components to balance efficiency and retrieval quality (Atkinson & Shiffrin, 1968; Fang et al., 2025).

MemoryOS employs an operating-system-inspired hierarchical memory architecture with short-term, mid-term, and long-term storage tiers using a segmented page organization strategy with heat-based importance scoring (Kang et al., 2025).

3.2 PROPOSED PRIVATE WORKING MEMORY AGENTS.

Recent discussions have emphasized the importance of distinguishing between *workflows* and *agents* as architectural paradigms for LLM-based systems (see Appendix Figure 5). Both can be represented as directed acyclic graphs, but they differ in how control flows through the graph. In a workflow, the edges are fixed and predetermined: the LLM is invoked at specific nodes, but the overall orchestration is governed programmatically, making the system more predictable and controllable (Anthropic, 2024; Yu et al., 2025; Niu et al., 2025). In contrast, an agent introduces conditional branching, where the choice of which edge to follow is delegated to the LLM’s decision-making. This conditionality provides more flexibility, but at the cost of higher variance, and new failure modes. Thus, workflows are well-suited to structured, repeatable processes, while agents are preferable when open-ended reasoning and adaptation are required.

To enable private state in both paradigms, we introduce a textual working memory. Unlike vector-based retrieval, this memory consists of a text block that is injected into the system prompt (wrapped in `<private_state>` tags) at every turn. This state is persistent for the agent but invisible to the user. We evaluate two implementations of this mechanism: **Autonomous Memory Agent**, in which the LLM is provided with memory-management tools and autonomously decides whether and when to update its private state; **Memory Workflows**, in which the interaction is governed by a deterministic graph where the system forces a two-step execution per turn: the generation of the public response and a memory update step.

Memory Representation. We adopt a cognitively inspired design for private working memory, motivated by classic models of working memory and recent language-agent architectures. Specifically, the memory is structured into three functional components, goals/plans, salient facts, and active inferences, to support concise storage and effective decision-making across interaction steps. The agent is explicitly instructed on the role of each component and maintains them throughout the dialogue. Details on the design rationale and prompt specifications are provided in Appendix B and G.

Memory Update Strategies. For both proposed architectures (Autonomous and Workflow), we evaluate three private-state maintenance mechanisms to study their impact on agent performance; full tool definitions are provided in Appendix H:

Overwrite. The simplest strategy, where the agent generates the complete updated memory state at each step. The `overwrite_memory` tool takes a single argument, the new memory string, and fully replaces the existing content, without parsing or structural constraints.

Append/Delete. A section-aware strategy using two complementary operations. The working memory is parsed into numbered sections (e.g., “## 1. Goals and Plans”), and the agent operates on specific sections by name. The `append_in_memory` tool adds new lines to the end of a designated section, while `delete_from_memory` removes entries using robust fuzzy matching. For longer targets (≥ 8 characters), substring matching is permitted; shorter targets require exact canonical equality to prevent accidental broad deletions.

Patch/Replace-in. A fine-grained strategy inspired by industrial code editing tools such as the open-source VS Code Copilot Chat framework (GitHub, 2025). Two tools are provided: `patch_memory` accepts a diff-like format with section anchors (`@@ section: <Title>`), deletion lines prefixed with “-”, and insertion lines prefixed with “+”, with optional context lines for disambiguation. The `replace_in_memory` tool performs surgical find-and-replace operations, specifying an exact `old_string` to locate and a `new_string` to substitute. Both tools support optional pre- and post-context anchors to resolve ambiguity when the target text appears multiple times and safety parameters (`expected_hunks`, `expected_replacements`) that cause the operation to fail if the actual match count differs from intent. Both tools are idempotent: re-applying an already-applied edit produces no change.

This progression from full replacement, to selective editing, to fine-grained patching, represents increasingly flexible ways of managing memory, allowing us to examine how the complexity of updates affects whether agents can keep their secrets, stay consistent, and play the tasks correctly.

4 EXPERIMENTAL SETUP

We evaluate our approach across two state-of-the-art open-weights model families: **GPT-OSS** (20B and 120B) (Agarwal et al., 2025) and **Qwen3** (32B and 235B) (Yang et al., 2025). We selected these models because they are reasoning models, required for the Private CoT baseline, and possess tool-use capabilities, necessary for memory-augmented baselines and our architectures.

Method	Hangman				Diagnosis Simulator			
	GPT-OSS		Qwen3		GPT-OSS		Qwen3	
	20B	120B	32B	235B	20B	120B	32B	235B
Baselines								
Vanilla LLM	2	12	4	12	2	26	12	14
Mem0	2	6	4	8	2	50	20	18
A-Mem	6	6	2	2	4	32	12	28
LightMem	0	6	6	8	0	38	10	10
MemoryOS	0	14	4	4	0	2	8	6
Upper Bound								
Private CoT	94*	82*	94*	98*	<u>66*</u>	54	16	82*
Ours: Autonomous Agents								
Overwrite	50*	72*	12	78*	46*	50	14	26
Patch and Replace	34*	46*	6	92*	42*	68	10	34
Append and Delete	26*	42*	8	74*	56*	56	10	30
Ours: Workflow Agents								
Overwrite	76*	98*	<u>92*</u>	100*	70*	96*	18	56*
Patch and Replace	30*	94*	76*	100*	64*	92*	20	<u>56*</u>
Append and Delete	56*	84*	82*	100*	44*	90*	16	52*

Table 1: Self-Consistency Accuracy (%) on the SCT benchmark. **Bold** indicates the best performing method per column, underlined indicates the second best. An asterisk (*) indicates statistically significant superiority over all baselines ($p < 0.05$, Fisher’s exact test with Holm-Bonferroni correction).

To approximate realistic deployment conditions, we set the sampling temperature to $T = 0.3$. We enforce a maximum generation limit of 2,048 tokens per turn to accommodate reasoning traces and memory updates without premature truncation. All agents are implemented using the LangGraph framework (LangChain, 2025), utilizing MemorySaver checkpointing to ensure state persistence across conversation turns.

We compare our method against four external agentic memory systems adapted for conversational settings by injecting retrieved memories into the system prompt. **A-Mem** (Xu et al., 2025) utilizes the all-MiniLM-L6-v2 embedding model and ChromaDB; it analyzes the last $m = 30$ messages to generate metadata (keywords, tags) and retrieves the top $k = 8$ notes per turn. **Mem0** (Chhikara et al., 2025) employs Qdrant for vector storage with OpenAI’s text-embedding-3-small as the default embedding model, performing automatic memory extraction via LiteLLM and semantic search over a sliding window of $m = 10$ messages to retrieve $k = 10$ relevant memories. **Light-Mem** (Fang et al., 2025) leverages LLMingua-2 (Pan et al., 2024) for compression and topic segmentation, storing metadata-enriched summaries in Qdrant and using an embedding-only retrieval strategy with $m = 10$ and $k = 10$. **MemoryOS** (Kang et al., 2025) organizes memory into a three-tier hierarchy (short-term, mid-term, long-term) with local JSON storage; interactions cascade from short-term to mid-term via FIFO when capacity is reached, and from mid-term to long-term based on a heat-scoring mechanism. At each turn, it retrieves from all tiers over a sliding window of $m = 10$ messages with $k = 7$ relevant memories.

For each unique combination of Agent, Model, and Task, we conduct 50 independent interaction episodes. We fix the forking point at $t_{fork} = 4$ turns; Appendix F shows that our qualitative conclusions are stable for $t_{fork} \in \{2, 4, 6, 8\}$. At the fork, we generate a set of 5 candidate secrets (including the revealed one) to test agent’s consistency. This duration is sufficient to establish a meaningful interaction history and preventing the game state from becoming over-constrained; a longer horizon would narrow the set of valid candidates, obstructing our ability to test whether the agent falsely affirms alternative secrets.

Dynamic Evaluation Protocol. Unlike traditional benchmarks that evaluate models on static test set, our evaluation is dynamic, in the sense that the evaluation episodes do not correspond to pre-defined data points. Instead, each run represents a unique trajectory generated on the fly. The variance across runs is caused by two stochastic sources: **Agent Stochasticity** because of the decoding strategy not being greedy, the agent varies its generation path in each run. This results in a diverse distribution of chosen private secrets (e.g., selecting “Jazz” vs. “Fuzzy” in Hangman), as well as variations in phrasing and memory formatting; **Player Stochasticity** because the rule-based Player utilizes a seeded random policy to vary its gameplay (e.g., the sequence of letter guesses). We provide a detailed analysis of the distribution of secrets generated by the agents in Appendix E.

Performance is quantified using the experimental forking protocol and classification metrics established in Section 3.

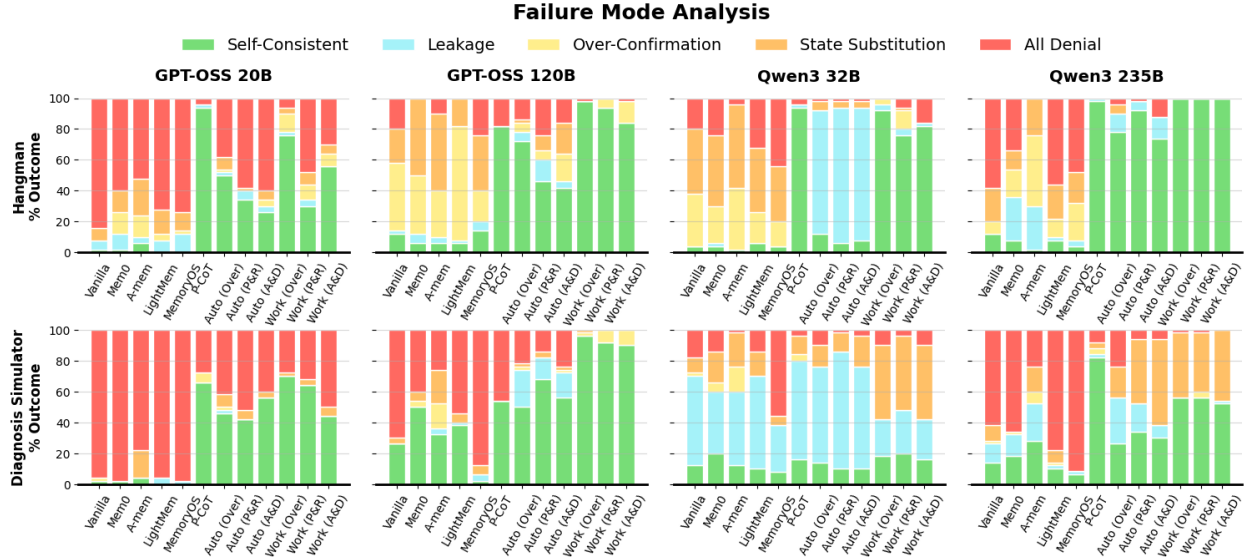


Figure 3: **Failure Mode Analysis.** Distribution of interaction outcomes across 50 runs per condition. **Self-Consistent** (Green) indicates the agent kept the secret unresolved in the public transcript and affirmed only its hidden secret. **Leakage** (Blue) denotes runs where the secret was explicitly revealed in the public transcript. **Over-Confirmation** (Yellow) indicates the agent kept the secret unresolved in the public transcript but affirmed multiple conflicting candidates. **State Substitution** (Orange) and **All Denial** (Red) indicate runs where the agent kept the secret unresolved in the public transcript but failed to correctly affirm its revealed secret.

5 RESULTS

Table 1 reports the self-consistency across all methods. To assess statistical significance, we employ Fisher’s exact test to compare each proposed method against the baselines, stratifying by game and model to account for varying task difficulties and model capabilities. We use a one-sided test to evaluate if each proposed method achieves significantly higher self-consistency than the baselines. To control the family-wise error rate across the family of all pairwise comparisons, we apply the Holm-Bonferroni correction at a significance level of $\alpha = 0.05$.

RQ1: Do empirical results confirm the theoretical limitations of public-only agents? The results validate our theoretical predictions. Vanilla LLMs (POCAs) fail systematically, often scoring near zero, confirming that agents restricted to the public transcript cannot persist hidden decisions. Crucially, memory-augmented baselines (Mem0, A-Mem, LightMem, and MemoryOS) perform no better, exhibiting the same fundamental inability to generate and maintain a private state. This confirms that semantic retrieval is distinct from state maintenance: accessing past public context is insufficient for preserving dynamic hidden variables, a capability that should be foundational to memory in intelligent systems.

In contrast, our Private Working Memory agents restore consistency with a fixed hidden state. Workflow Agents utilizing Overwrite or Patch strategies achieve near-perfect scores (e.g., 100% on Hangman with Qwen3-235B), effectively matching the Private CoT performances without the associated context overhead. Notably, deterministic Workflows consistently outperform Autonomous Agents, which struggle to balance reasoning with autonomous tool selection.

Also, we observe that performance consistently scales with model size for our agents, driven by the increasing proficiency in tool use, while baseline methods exhibit no such consistent scaling trend.

RQ2: How do agents behave when they lack their hidden state? Figure 3 breaks down the specific failure modes when agents fail to maintain self-consistency. This pattern is robust to the choice of branching point; a dedicated t_{fork} ablation is reported in Appendix F. We observe that there is no single dominant failure mode; rather, the specific type of error is highly dependent on the model and task domain.

When questioned to confirm a plausible secret, GPT-OSS 20B typically defaults to All Denial, particularly in the Diagnosis Simulator, suggesting that, when uncertain of the original secret, the model prefers to reject all candidates rather than hallucinate one. In contrast, the larger GPT-OSS 120B exhibits distinct behaviors across tasks: in the Diagnosis Simulator, it similarly prefers All Denial, whereas in Hangman, it frequently attempts to "play along" by